

Curso: Engenharia de Software

Disciplina: Teste de Software

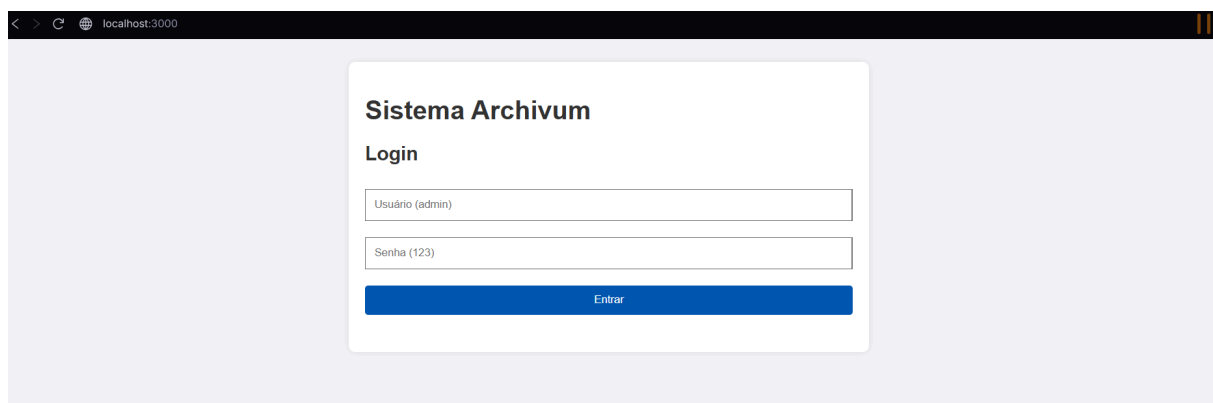
Alunos: Alexandre Rodrigues Costa, Caio Murilo Silva de Oliveira, Caian Dieter Passos, Ester Yasmin Santos Silva, André Luiz Borges Aguiar e Artur Braz Lopes

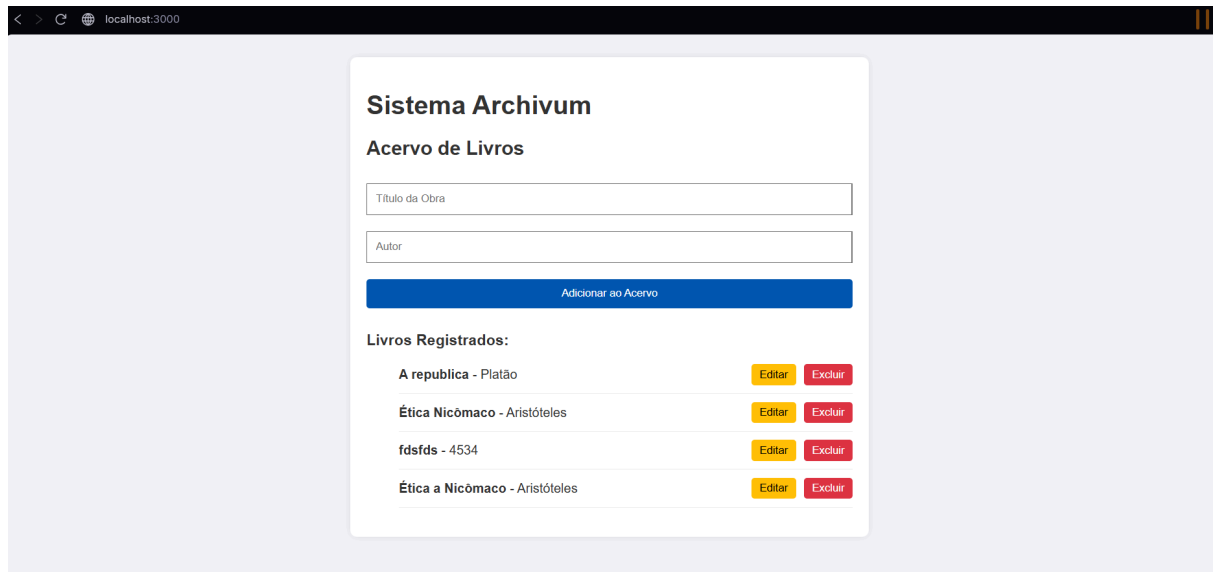
Relatório Final de Teste de Software: Sistema Archivum

1. Introdução ao sistema desenvolvido

O sistema escolhido pelo nosso grupo para o desenvolvimento e aplicação das práticas de teste foi o Archivum. Trata-se de uma aplicação web voltada para a área da Educação , projetada para permitir que estudantes façam o fichamento e o gerenciamento do seu acervo de obras literárias de forma simples e direta.

O projeto foi construído utilizando uma arquitetura modular em Python. No backend, utilizamos o framework Flask para construir a API e lidar com as rotas. Para a persistência de dados, implementamos um banco de dados SQLite (archivum.db). A interface do usuário (frontend) foi desenvolvida em HTML, CSS e JavaScript (Vanilla) , comunicando-se diretamente com a API para realizar o controle de acesso (login) e o CRUD completo das obras.





2. Documentação de Requisitos

Documento de Visão: O Archivum visa fornecer uma plataforma minimalista e funcional onde usuários podem cadastrar, visualizar, editar e excluir informações sobre livros lidos ou artigos estudados, garantindo a persistência dos dados através de uma API RESTful.

Histórias de Usuário (BDD):

- **Cenário 1: Autenticação no Sistema**
 - Dado que o usuário está na tela inicial de acesso;
 - Quando ele insere as credenciais válidas e clica em "Entrar";
 - Então o sistema oculta a tela de login e exibe a interface do acervo de livros.
- **Cenário 2: Adição de nova obra (Create)**
 - Dado que o usuário está logado na interface do acervo;
 - Quando ele preenche os campos "Título" e "Autor" e aciona o botão de adicionar;
 - Então os dados são enviados para a API, salvos no banco SQLite e a lista na tela é atualizada imediatamente.

3. Planejamento de Testes

Objetivo dos testes: Validar a estabilidade das rotas da API (backend) e garantir que a integração entre a interface do usuário e o banco de dados ocorra sem falhas durante o uso comum do sistema.

Escopo do teste: Serão testados o fluxo de autenticação e as quatro operações do CRUD (Create, Read, Update, Delete). Estão fora do escopo testes de carga, estresse e validações de segurança complexas.

Ferramentas utilizadas: Para a automação, utilizamos o Pytest (testes unitários e de API) e o Playwright (testes End-to-End simulando o navegador).

Estratégia de testes: A abordagem utilizada foca na criação de um banco de dados temporário em memória durante os testes da API, garantindo o isolamento dos dados. Para o E2E, a estratégia foca no comportamento do DOM após interações do usuário na tela.

Papéis e Responsabilidades:

- **Caio Murilo:** Implementação da documentação e auxílio no código.
- **Alexandre:** Auxílio na implementação do código.
- **Caian:** Auxílio na implementação do código.
- **Ester:** Auxílio na implementação do código.
- **André:** Revisão do código e formatação do relatório final.
- **Artur:** Revisão do código e formatação do relatório final.

4. Descrição dos Casos de Teste

CT-01: Validação de Login com Sucesso (API)

- **Descrição:** Verifica se a rota /api/login permite o acesso com as credenciais corretas.
- **Passos:** Iniciar o client de teste -> Disparar um POST para a rota de login com o payload de usuário -> Verificar o Status Code da resposta.
- **Dados de entrada:** {"usuario": "admin", "senha": "123"}
- **Saída esperada:** Token de autenticação.

- **Critério de sucesso:** Retorno de HTTP Status 200 e payload {"token": "autenticado"}.

CT-02: Execução Completa do CRUD (API)

- **Descrição:** Valida o ciclo de vida do dado na rota /api/livros.
- **Passos:** Realizar um POST (Criar obra) -> GET (Ler a lista) -> PUT (Editar a obra criada) -> DELETE (Remover a obra).
- **Dados de entrada:** {"titulo": "O Príncipe", "autor": "Maquiavel"}.
- **Saída esperada:** Confirmação em formato JSON para cada operação.
- **Critério de sucesso:** O sistema deve retornar Status 201 no POST, e Status 200 nas demais operações. O id do livro deve ser mantido durante o fluxo.

CT-03: Fluxo E2E de Login e Cadastro (Interface)

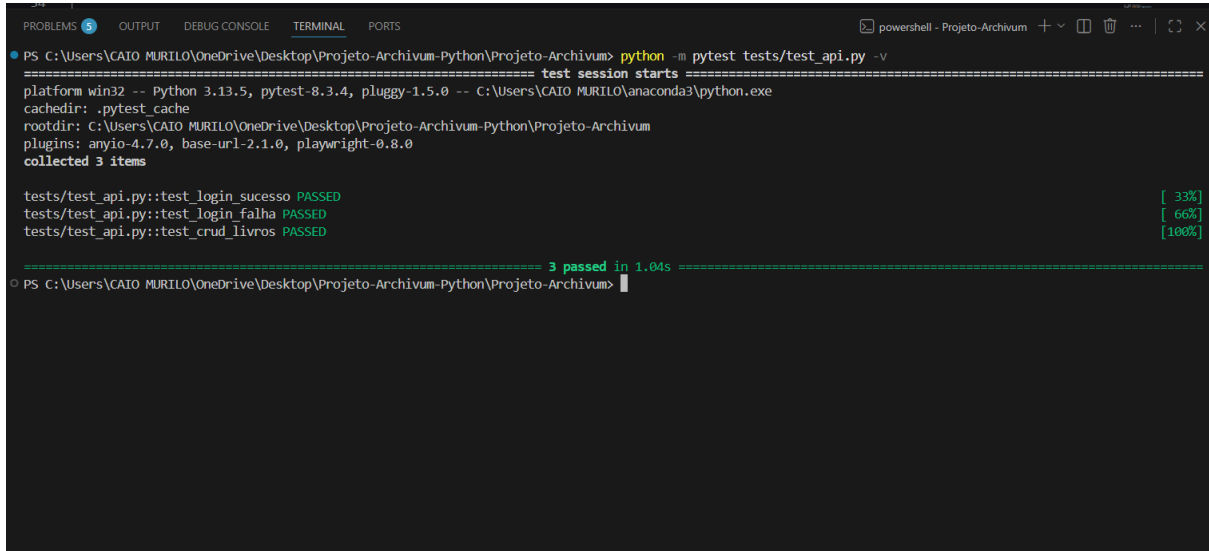
- **Descrição:** Simula um usuário abrindo o navegador, logando e registrando um livro.
- **Passos:** Acessar localhost:3000 -> Preencher inputs de login -> Clicar em Entrar -> Preencher inputs de livro -> Clicar em Adicionar -> Verificar tela.
- **Dados de entrada:** Credenciais admin/123 e Livro "Ética a Nicômaco" / "Aristóteles".
- **Saída esperada:** O nome do livro deve aparecer renderizado na lista HTML na tela.
- **Critério de sucesso:** O elemento visual #listaLivros deve conter o texto inserido sem gerar exceções no console.

5. Resultados das Execuções e Evidências

A execução da nossa suíte de testes não apresentou falhas, bugs ou gargalos. O sistema se comportou de maneira estável tanto no backend quanto no frontend. Abaixo estão as evidências:

Evidência 01: Relatório de Execução da API (Pytest)

(O log demonstra os 3 testes unitários/API processados com sucesso em menos de 1 segundo).



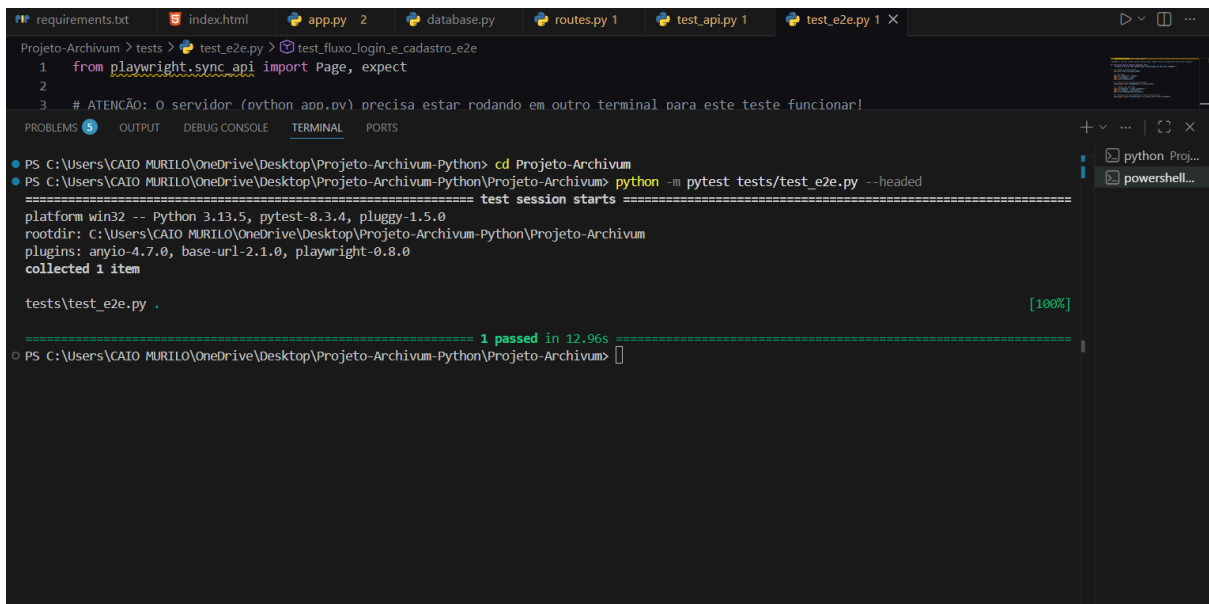
```
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum> python -m pytest tests/test_api.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-8.3.4, pluggy-1.5.0 -- C:\Users\CAIO MURILO\anaconda3\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum
plugins: anyio-4.7.0, base-url-2.1.0, playwright-0.8.0
collected 3 items

tests/test_api.py::test_login_sucesso PASSED [ 33%]
tests/test_api.py::test_login_falha PASSED [ 66%]
tests/test_api.py::test_crud_livros PASSED [100%]

===== 3 passed in 1.04s =====
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum>
```

Evidência 02: Relatório de Execução E2E (Playwright)

(O log confirma que o robô conseguiu interagir com o navegador, logar e cadastrar os dados).

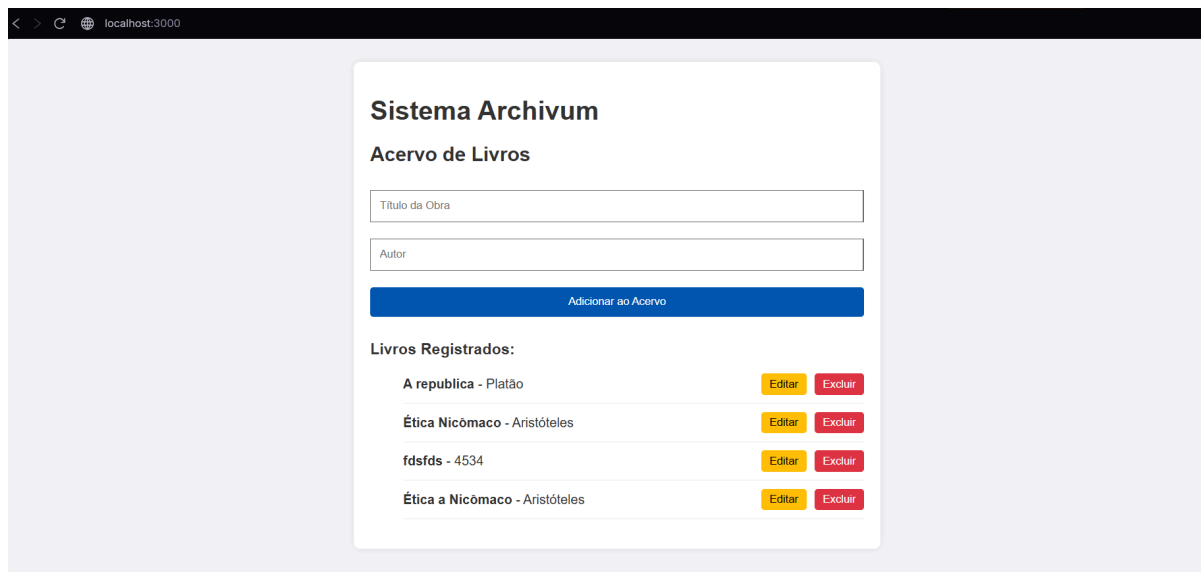


```
Projeto-Archivum > tests > test_e2e.py > test_fluxo_login_e_cadastro_e2e
1 from playwright.sync_api import Page, expect
2
3 # ATENÇÃO: O servidor (uvrton app.py) precisa estar rodando em outro terminal para este teste funcionar!

PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum> cd Projeto-Archivum
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum> python -m pytest tests/test_e2e.py --headed
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-8.3.4, pluggy-1.5.0
rootdir: C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum
plugins: anyio-4.7.0, base-url-2.1.0, playwright-0.8.0
collected 1 item

tests\test_e2e.py . [100%]

===== 1 passed in 12.96s =====
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum>
```



6. Conclusão e Lições Aprendidas

O desenvolvimento prático desta atividade nos trouxe uma grande percepção sobre decisões de arquitetura e Engenharia de Software. Inicialmente, cogitamos construir o sistema em JavaScript (Node.js). Contudo, percebemos que para focar na qualidade dos testes e nas ferramentas que o grupo tinha mais domínio, refatorar a base de código para Python seria a escolha ideal.

Essa decisão estratégica nos permitiu usar o Pytest de forma nativa e limpa, facilitando a injeção do banco de dados temporário. Aprendemos a importância vital de separar as responsabilidades do código (Modularização com `routes.py` e `database.py`), pois um código monolítico torna-se extremamente complexo de ser testado automaticamente. A aplicação das técnicas E2E demonstrou na prática como garantir a qualidade do que é entregue ao cliente final.

7. Link para o Repositório no GitHub

O código-fonte completo da aplicação, bem como a suíte de testes desenvolvida e o `requirements.txt` para fácil instalação, estão hospedados no link abaixo:

<https://github.com/CaioMurilo/projeto-archivum-testes/tree/main>